Syllabus

Inheritance:

- Base Class and derived Class, access specifiers, Constructor and Destructor in Derived Class,
- Virtual destructor, Protected members, Overriding member functions, Public and Private
- Inheritance, Multiple Inheritance, Ambiguity in Multiple Inheritance, Composition, Nested
- Classes

inheritance

- Inheritance is a process in which one *object acquires all the properties and behaviors of its parent object* automatically.
- In such way, you can reuse, extend or modify the attributes and behaviors which are defined in other class.
- In C++, the class which inherits the members of another class is called *derived class* and
- The class whose members are inherited is called *base class.*
- The derived class is the specialized class for the base class.

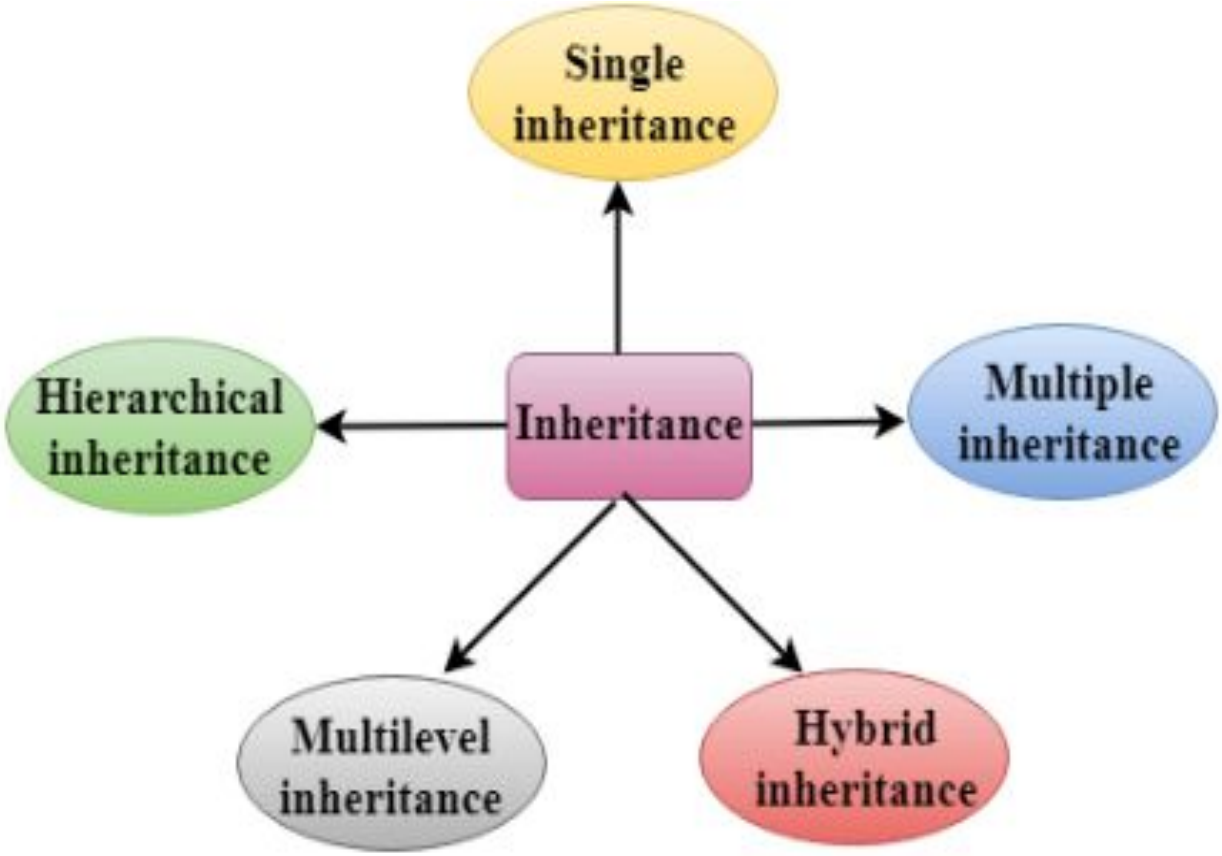
Syntax

```
class derived_class_name : visibility mode base_class_name
{
   // body of the derived class.
}
```

Where,

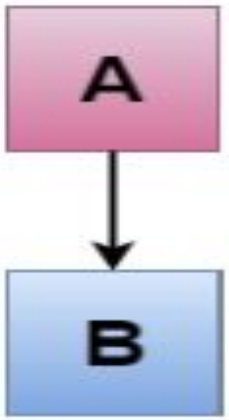
- **derived_class_name:** It is the name of the derived class.
- **visibility mode:** The visibility mode specifies whether the features of the base class are publicly inherited or privately inherited. It can be public or private.
- **base_class_name:** It is the name of the base class.
- When the base class is privately inherited by the derived class, public members of the base class becomes the private members of the derived class. Therefore, the public members of the base class are not accessible by the objects of the derived class only by the member functions of the derived class.
- When the base class is publicly inherited by the derived class, public members of the base class also become the public members of the derived class. Therefore, the public members of the base class are accessible by the objects of the derived class as well as by the member functions of the base class.

Types


Single inheritance
Hierarchical inheritance
Inheritance
Multiple inheritance
Multilevel inheritance
Hybrid inheritance

Single inheritance

- **Single inheritance** is defined as the inheritance in which *a derived a new class from the existing only one base class*.



Where 'A' is the base class, and 'B' is the derived class.

Program to demonstrate single inheritance

```
#include <iostream>
using namespace std;
class Parent
{
   int a = 4;
   int b = 5;
   public:
   int mul()
   {
      int c = a*b;
      return c;
   }
};
class derived : private Parent
{
   public:
   void display()
   {
      int result = mul();
      cout <<"Multiplication of a and b is : "<<result<< endl;
   }
};
int main()
{
  derived  d;
  d.display();
    return 0;
}
```

Output:

```
Multiplication of a and b is : 20
```

```
// C++ program to demonstrate how to implement the Single
// inheritance
#include <iostream>
using namespace std;
class Vehicle                          // base class
{
public:
   Vehicle() { cout << "This is a Vehicle\n"; }
};

class Car : public Vehicle      // Derive class
{
public:  Car()
            {
                cout << "This Vehicle is Car\n";
             }
};

int main()
{
   // Creating object of sub class will  invoke the constructor of base classes
   Car obj;
   return 0;
}
```

Multilevel Inheritance

- Multilevel Inheritance
- **Multilevel inheritance** is a process of deriving a class from another derived class.

```
// Program to demonstrate multilevel inheritance with real life example
#include <iostream>
using namespace std;
 class Animal {                          // Base class
   public:
          void eat() {
            cout<<"Eating..."<<endl;
 }
   };
   class Dog: public Animal      // derive class Dog from base class Animal
   {
      public:
     void bark(){
    cout<<"Barking..."<<endl;
     }
   };
   class BabyDog: public Dog    // derive class BabyDog from base class Dog
   {
      public:
     void weep() {
    cout<<"Weeping...";
     }
   };
int main(void) {
    BabyDog d1;
    d1.eat();          // function of animal class
    d1.bark();       // function of Dog class
     d1.weep();    // function of BabyDog class
     return 0;
}
```

Output:

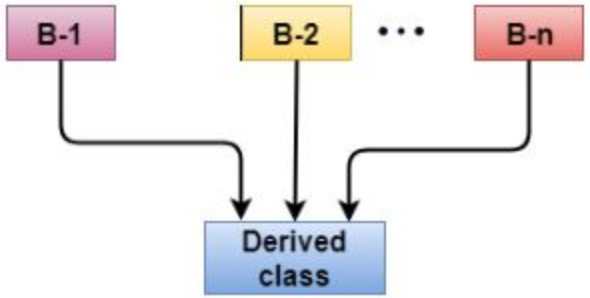
```
Eating...
Barking...
Weeping...
```

```
// C++ program to implement Multilevel Inheritance
#include <iostream>
using namespace std;

// base class
class Vehicle {
public:
    Vehicle() { cout << "This is a Vehicle\n"; }
};

// first sub_class derived from class vehicle
class fourWheeler : public Vehicle {
public:
    fourWheeler() { cout << "4 Wheeler Vehicles\n"; }
};

// sub class derived from the derived base class fourWheeler
class Car : public fourWheeler {
public:
    Car() { cout << "This 4 Wheeler Vehical is a Car\n"; }
};

// main function
int main()
{
    // Creating object of sub class will
    // invoke the constructor of base classes.
    Car obj;
    return 0;
}
```

Multiple inheritance

- Its process of deriving a *new class that inherits the attributes from two or more base classes.*
- *Syntax*

```
class D : visibility B-1, visibility B-2
{
   // Body of the class;
}
```



```
// Program to demonstrate multiple Inheritance
#include <iostream>
using namespace std;
class A
{
   protected:
    int a;
   public:
   void get_a(int n)
   {
      a = n;
   }
};

class B
{
   protected:
   int b;
   public:
   void get_b(int n)
   {
      b = n;
   }
};
class C : public A,public B
{
  public:
   void display()
   {
      std::cout << "The value of a is : " <<a<< std::endl;
      std::cout << "The value of b is : " <<b<< std::endl;
      cout<<"Addition of a and b is : "<<a+b;
   }
};
int main()
{
  C c;
  c.get_a(10);
  c.get_b(20);
  c.display();

   return 0;
}
```

Ambiguity in Inheritance

```
#include <iostream>
using namespace std;
class Amar
{
   public:
   void display()
   {
      std::cout << "Class Amar Data" << std::endl;
   }
};
class Agabar
{
   public:
   void display()
   {
      std::cout << "Class  Agabar Data " << std::endl;
   }
};
class Anthony : public  Amar, public Agabar
{
   void view()
   {
      display();
   }
};
int main()
{
   C c;
   c.display();
   return 0;
}
```

```
error: reference to 'display' is ambiguous
        display();
```

Solution :

```
class Anthony : public  Amar, public Agabar
{
    void view()
    {
  Amar  :: display();   // Calling the display() function of class A
  Agabar:: display();  // Calling the display() function of class B.
    }
};
```

```
// C++ program to illustrate the multiple inheritance
#include <iostream>
using namespace std;
class Vehicle {                  // first base class
public:
   Vehicle()
                  {
                        cout << "This is a Vehicle\n";
                   }
};

// second base class
class FourWheeler {
public:
   FourWheeler() { cout << "This is a 4 Wheeler\n"; }
};
// sub class derived from two base classes
class Car : public Vehicle, public FourWheeler {
  public:
   Car() { cout << "This 4 Wheeler Vehical is a Car\n"; }
};

// main function
int main()
{
   // Creating object of sub class will  invoke the constructor of base classes.
   Car obj;
   return 0;
}
```

Hierarchical Inheritance

- In this type of inheritance, more than one subclass is inherited from a single base class. i.e. more than one derived class is created from a single base class.

- Synatx

```
class derived_class1: access_specifier base_class
    {
    ... .. ...
    }
class derived_class2: access_specifier base_class
    {
    ... .. ...
    }
```

```
// C++ program to implement Hierarchical Inheritance
#include <iostream>
using namespace std;
// base class
class Vehicle {
public:
    Vehicle() { cout << "This is a Vehicle\n"; }
};
// first sub class
class Car : public Vehicle {
public:
    Car() { cout << "This Vehicle is Car\n"; }
};
// second sub class
class Bus : public Vehicle {
public:
    Bus() { cout << "This Vehicle is Bus\n"; }
};

// main function
int main()
{
    // Creating object of sub class will invoke the constructor of base class.
    Car obj1;
    Bus obj2;
    return 0;
}
```

Hybrid Inheritance

- Hybrid Inheritance is implemented by combining more than one type of inheritance. For example: Combining Hierarchical inheritance and Multiple Inheritance will create hybrid inheritance in C++
- There is no particular syntax of hybrid inheritance.
- We can just combine two of the above inheritance types

```
// C++ program to illustrate the implementation of Hybrid Inheritance
#include <iostream>
using namespace std;

// base class
class Vehicle {
public:
    Vehicle() { cout << "This is a Vehicle\n"; }
};

// base class
class Fare {
public:
    Fare() { cout << "Fare of Vehicle\n"; }
};

// first sub class
class Car : public Vehicle {
  public:
  Car() { cout << "This Vehical is a Car\n"; }
};

// second sub class
class Bus : public Vehicle, public Fare {
  public:
  Bus() { cout << "This Vehicle is a Bus with Fare\n"; }
};

// main function
int main()
{
    // Creating object of sub class will
    // invoke the constructor of base class.
    Bus obj2;
    return 0;
}
```

public inheritance

- makes public members of the base class *public* in the derived class,
- and the protected members of the base class remain protected in the derived class

```
// C++ program to demonstrate the working of public inheritance

#include <iostream>
using namespace std;

class Base {
  private:
    int pvt = 1;

  protected:
    int prot = 2;

  public:
    int pub = 3;

    // function to access private member
    int getPVT() {
      return pvt;
    }
};

class PublicDerived : public Base {
  public:
    // function to access protected member from Base
    int getProt() {
      return prot;
    }
};

int main() {
  PublicDerived object1;
  cout << "Private = " << object1.getPVT() << endl;
  cout << "Protected = " << object1.getProt() << endl;
  cout << "Public = " << object1.pub << endl;
  return 0;
}
```